

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL XII
SOLID PRINCIPLE



Disusun Oleh:

Neila Faaizah Asynur 105224028

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. Pendahuluan

Praktikum modul 12 membahas tentang Solid Principle. Solid merupakan lima prinsip dasar desain berorientasi objek (OOP) yang terdiri dari:

- S (Single Responsibility): Memecah kelas raksasa agar satu kelas hanya memiliki satu tanggung jawab spesifik.
- O (Open/Closed): Membuat kode terbuka untuk diperluas tetapi tertutup untuk memodifikasi kode lama.
- L (Liskov Substitution): Kelas turunan harus bisa menggantikan tipe dasarnya tanpa merusak tipe dasar tersebut.
- I (Interface Segregation): Memisahkan interface umum agar tidak ada error paksaan pada kelas yang tidak mengimplementasikan sebuah interface.
- D (Dependency Inversion): Memutuskan ketergantungan langsung pada database konkret agar sistem siap dimigrasikan ke Cloud NoSQL kapan saja tanpa merusak inti sistem.

Dalam penerapan di dunia nyata, kelima prinsip SOLID tersebut digunakan untuk mendesain ulang arsitektur sistem informasi akademik (SIKAD) yang mengalami pembusukan kode. Melalui restrukturisasi ini, diharapkan kode program SIKAD ditransformasikan menjadi bahasa pemrograman Java yang lebih tangkas.

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1.	nim	String	Menyimpan nim dari setiap mahasiswa
2.	nama	String	Menyimpan nama dari mahasiswa
3.	tipeJalur	String	Menyimpan tipe jalur masuk dari setiap mahasiswa
4.	kode	String	Menyimpan kode unik dari setiap mata kuliah
5.	nama	String	Meyimpan nama dari mata kuliah
6.	sks	Int	Menyimpan jumlah sks dari mata kuliah yang dipilih
7.	daftarMataKuliah	List<Mata Kuliah>	Menampung value dari mata kuliah yang diambil oleh mahasiswa
8.	tarifInternasional	double	Menyimpan besaran nilai tarif yang harus dibayarkan untuk tipe jalur internasional

9.	tarifJurusan	double	Menyimpan besaran nilai tarif yang harus dibayarkan dari suatu jurusan
10.	biayaKelasMalam	double	Menyimpan besaran nilai yang harus dibayarkan untuk mengikuti kelas malam
11.	mhs1, mhs2, mhs3	Mahasiswa	Objek yang berguna untuk menampung data identitas dari seorang mahasiswa
12.	krsNeila	KRS	Objek yang berguna untuk menyimpan nilai dari krs yang diambil oleh Neila
13.	praktikumPbo	MataKuliahPraktikum	Objek yang berguna untuk menyimpan informasi terkait praktikum pbo
14.	uktNeila	UKTReguler	Menyimpan besaran UKT yang harus dibayarkan untuk tipe reguler
15.	krsAtin	KRS	Objek yang berguna untuk menyimpan nilai dari krs yang diambil oleh Atin
16.	uktAtin	UKTKaryawan	Menyimpan besaran UKT yang harus dibayarkan untuk tipe karyawan
17.	krsIya	KRS	Objek yang berguna untuk menyimpan nilai dari krs yang diambil oleh Iya
18.	uktIya	UKTBidikMisi	Menyimpan besaran UKT yang harus dibayarkan untuk tipe bidik misi

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1.	Mahasiswa()	Constructor	Inisialisasi awal objek Mahasiswa ketika pertama kali membuat objek

2.	getNim()	Procedural	Getter yang berguna untuk mengambil value dari atribut nim yang terenkapsulasi
3.	getNama()	Procedural	Getter yang berguna untuk mengambil value dari atribut nama yang terenkapsulasi
4.	getTipeJalur())	Procedural	Getter yang berguna untuk mengambil value dari atribut tipe jalur yang terenkapsulasi
5.	getKode()	Procedural	Getter yang berguna untuk mengambil kode dari suatu mata kuliah
6.	getNama()	Procedural	Getter yang berguna untuk mengambil nama dari suatu mata kuliah
7.	getSks()	Procedural	Getter yang berguna untuk mengambil jumlah sks dari suatu mata kuliah
8.	validasiPrasyarat(String nim)	Precedural	Melakukan pengecekan syarat terhadap suatu mata kuliah
9.	MataKuliahKKN(String kode, String nama, int sks)	Constructor	Inisialisasi awal objek MataKuliahKKN ketika pertama kali membuat objek
10	MataKuliahMBKM(String kode, String nama, int sks)	Constructor	Inisialisasi awal objek MataKuliahMBKM ketika pertama kali membuat objek
11.	MataKuliahPraktikum(String kode, String nama, int sks)	Constructor	Inisialisasi awal objek MataKuliahPraktikum ketika pertama kali membuat objek
12.	alokasiAsistenLab()	Procedural	Melakukan alokasi terhadap asisten lab ke sebuah lab
13.	cekPeralatanPraktikum()	Procedural	Melakukan pengecekan terhadap alat praktikum

14.	MataKuliahTeori(String kode, String nama, int sks)	Constructor	Inisialisasi awal objek MataKuliahTeori ketika pertama kali membuat objek
15.	hitungUKT()	Procedural	Menghitung nominal UKT yang harus dibayarkan oleh seorang Mahasiswa
16.	UKTBidikMisi	Konstruktor	Inisialisasi awal objek UKTBidikMisi ketika pertama kali membuat objek
17.	UKTInternasional	Konstruktor	Inisialisasi awal objek UKTInternasional ketika pertama kali membuat objek
18.	UKTkaryawan	Konstruktor	Inisialisasi awal objek UKTkaryawan ketika pertama kali membuat objek
19.	UKTReguler	Konstruktor	Inisialisasi awal objek UKTReguler ketika pertama kali membuat objek
20.	getDaftarMataKuliah	Procedural	Mengambil value yang terdapat di dalam List daftar mata kuliah
21.	main()	Procedural	Menjalankan dan mengeksekusi seluruh simulasi program

IV. Dokumentasi dan Pembahasan Code

Dokumentasi class KRS

```
import java.util.*;

public class KRS {
    private List<MataKuliah> daftarMataKuliah;

    public KRS(){
        this.daftarMataKuliah = new ArrayList<>();
    }

    public void tambahMataKuliah(MataKuliah matkul){
        this.daftarMataKuliah.add(matkul);
    }

    public List<MataKuliah> getDaftarMataKuliah(){
        return this.daftarMataKuliah;
    }
}
```

Gambar 1. Dokumentasi kode pada class KRS

Dokumentasi class Mahasiswa

```
public class Mahasiswa {
    private String nim;
    private String nama;
    private String tipeJalur;

    public Mahasiswa (String nim, String nama, String tipeJalur){
        this.nim = nim;
        this.nama = nama;
        this.tipeJalur = tipeJalur;
    }

    public String getNim (){
        return nim;
    }
    public String getNama(){
        return nama;
    }
    public String getTipeJalur(){
        return tipeJalur;
    }
}
```

Gambar 2. Dokumentasi pada class Mahasiswa

Dokumentasi class MataKuliahKKN

```
public class MataKuliahKKN implements MataKuliah{
    private String kode;
    private String nama;
    private int sks;

    public MataKuliahKKN(String kode, String nama, int sks){
        this.kode = kode;
        this.nama = nama;
        this.sks = sks;
    }

    @Override
    public String getKode(){
        return this.kode;
    }
    public String getNama(){
        return this.nama;
    }
    public int getSks(){
        return this.sks;
    }
    public boolean validasiPrasyarat(String nim){
        System.out.println("Memeriksa prasyarat teori untuk mata kuliah " + this.nama);
        return true;
    }
}
```

Gambar 3. Dokumentasi pada class MataKuliahKKN

Dokumentasi class MataKuliahMBKM

```
public class MataKuliahMBKM implements MataKuliah{
    private String kode;
    private String nama;
    private int sks;

    public MataKuliahMBKM(String kode, String nama, int sks){
        this.kode = kode;
        this.nama = nama;
        this.sks = sks;
    }

    @Override
    public String getKode(){
        return this.kode;
    }
    public String getNama(){
        return this.nama;
    }
    public int getSks(){
        return this.sks;
    }
    public boolean validasiPrasyarat(String nim){
        System.out.println("Memeriksa prasyarat teori untuk mata kuliah " + this.nama);
        return true;
    }
}
```

Gambar 4. Dokumentasi pada class MataKuliahMBKM

Dokumentasi class MataKuliahPraktikum

```
public class MataKuliahPraktikum implements KebutuhanPraktikum, MataKuliah{
    private String kode;
    private String nama;
    private int sks;

    public MataKuliahPraktikum(String kode, String nama, int sks){
        this.kode = kode;
        this.nama = nama;
        this.sks = sks;
    }

    @Override
    public String getKode(){
        return this.kode;
    }
    public String getNama(){
        return this.nama;
    }
    public int getSks(){
        return this.sks;
    }
    public boolean validasiPrasyarat(String nim){
        System.out.println("Memeriksa prasyarat teori untuk mata kuliah " + this.nama);
        return true;
    }

    public void alokasiAsistenLab(){
        System.out.println("Mengalokasikan asisten lab untuk praktikum " + this.nama);
    }
    public void cekPeralatanPraktikum(){
        System.out.println("Memeriksa alat untuk praktikum " + this.nama);
    }
}
```

Gambar 5. Dokumentasi pada class MataKuliahPraktikum

Dokumentasi class MataKuliahTeori

```
public class MataKuliahTeori implements MataKuliah{
    private String kode;
    private String nama;
    private int sks;

    public MataKuliahTeori(String kode, String nama, int sks){
        this.kode = kode;
        this.nama = nama;
        this.sks = sks;
    }

    @Override
    public String getKode(){
        return this.kode;
    }
    public String getNama(){
        return this.nama;
    }
    public int getSks(){
        return this.sks;
    }
    public boolean validasiPrasyarat(String nim){
        System.out.println("Memeriksa prasyarat teori untuk mata kuliah " + this.nama);
        return true;
    }
}
```

Gambar 6. Dokumentasi pada class MataKuliahTeori

Dokumentasi class UKTBidikMisi

```
public class UKTBidikMisi implements PenghitunganUKT{
    public UKTBidikMisi (){
    }

    @Override
    public double hitungUKT (){
        System.out.println(x: "Bayar ukt untuk kelas bidik misi");
        return 0.0;
    }
}
```

Gambar 7. Dokumentasi pada class UKTBidikMisi

Dokumentasi class UKTInternasional

```
public class UKTInternasional implements PenghitunganUKT{
    private double tarifInternasional;

    public UKTInternasional(double tarifInternasional){
        this.tarifInternasional = tarifInternasional;
    }

    @Override
    public double hitungUKT (){
        System.out.println(x: "Bayar ukt untuk kelas internasional");
        return this.tarifInternasional;
    }
}
```

Gambar 8. Dokumentasi pada class UKTInternasional

Dokumentasi class UKTKaryawan

```
public class UKTKaryawan implements PenghitunganUKT {
    private double tarifJurusan;
    private double biayaKelasMalam;

    public UKTKaryawan(double tarifJurusan, double biayaKelasMalam){
        this.tarifJurusan = tarifJurusan;
    }

    @Override
    public double hitungUKT (){
        System.out.println(x: "Bayar ukt untuk kelas karyawan");
        return this.tarifJurusan + this.biayaKelasMalam;
    }
}
```

Gambar 8. Dokumentasi pada class UKTKaryawan

Dokumentasi class UKTReguler

```
public class UKTReguler implements PenghitunganUKT{
    private double tarifJurusan;

    public UKTReguler (double tarifJurusan){
        this.tarifJurusan = tarifJurusan;
    }

    @Override
    public double hitungUKT (){
        System.out.println(x: "Bayar ukt untuk kelas reguler");
        return this.tarifJurusan;
    }
}
```

Gambar 9. Dokumentasi pada class UKTReguler

Dokumentasi main program (App)

```
public class App {
    Run | Debug
    public static void main(String[] args) {
        Mahasiswa mhs1 = new Mahasiswa(nim: "105224028", nama: "Neila Asynur", tipeJalur: "REGULER");

        KRS krsNeila = new KRS();
        krsNeila.tambahMataKuliah(new MataKuliahTeori(kode: "CS201", nama: "Struktur Data", sks: 3));

        MataKuliahPraktikum praktikumPbo = new MataKuliahPraktikum(kode: "CS201L", nama: "Praktikum PBO", sks: 1);
        krsNeila.tambahMataKuliah(praktikumPbo);

        System.out.println(" >> Memproses KRS untuk: " + mhs1.getNama());

        for (MataKuliah matkul : krsNeila.getDaftarMataKuliah()) {
            matkul.validasiPrasyarat(mhs1.getNim());

            if (matkul instanceof KebutuhanPraktikum) {
                KebutuhanPraktikum prak = (KebutuhanPraktikum) matkul;
                prak.cekPeralatanPraktikum();
                prak.alokasiAsistenLab();
            }
        }

        PenghitunganUKT uktNeila = new UKTReguler(tarifJurusan: 8000000.0);
        double totalUKTNeila = uktNeila.hitungUKT();
        System.out.println("Total Biaya yang harus dibayar " + mhs1.getNama() + " Rp" + totalUKTNeila);
        System.out.println(x: "-----\n");

        Mahasiswa mhs2 = new Mahasiswa(nim: "105224019", nama: "Fatin Atin", tipeJalur: "KARYAWAN");

        KRS krsAtin = new KRS();
        krsAtin.tambahMataKuliah(new MataKuliahTeori(kode: "AK101", nama: "Pengantar Akuntansi", sks: 3));

        System.out.println(" >> Memproses KRS untuk: " + mhs2.getNama());

        for (MataKuliah matkul : krsAtin.getDaftarMataKuliah()) {
            matkul.validasiPrasyarat(mhs2.getNim());
        }

        PenghitunganUKT uktAtin = new UKTKaryawan(tarifJurusan: 5000000.0, biayaKelasMalam: 1500000.0);
        double totalUKTAtin = uktAtin.hitungUKT();
        System.out.println("Total Biaya yang harus dibayar" + mhs2.getNama() + ": Rp" + totalUKTAtin);
        System.out.println(x: "-----\n");

        Mahasiswa mhs3 = new Mahasiswa(nim: "105224050", nama: "Iya Hayatul", tipeJalur: "MBKM");

        KRS krsIya = new KRS();

        krsIya.tambahMataKuliah(new MataKuliahMBKM(kode: "MBKM01", nama: "Magang Bersertifikat", sks: 6));

        System.out.println(" >> Memproses KRS untuk: " + mhs3.getNama());

        for (MataKuliah matkul : krsIya.getDaftarMataKuliah()) {
            matkul.validasiPrasyarat(mhs3.getNim());
        }

        PenghitunganUKT uktIya = new UKTBidikMisi();
        double totalUKTIya = uktIya.hitungUKT();
        System.out.println("Total Biaya yang harus dibayar " + mhs3.getNama() + ": Rp" + totalUKTIya);
        System.out.println(x: "-----\n");
    }
}
```

Gambar 10. Dokumentasi pada main program

Dokumentasi interface MataKuliah

```
1 public interface MataKuliah {
2     String getKode();
3     String getNama();
4     int getsks();
5
6     boolean validasiPrasyarat (String nim);
7 }
```

Gambar 11. Dokumentasi pada interface MataKuliah

Dokumentasi interface MataKuliah

```
public interface PenghitunganUKT {  
    public double hitungUKT();  
}
```

Gambar 12. Dokumentasi pada interface MataKuliah

Dokumentasi interface KebutuhanPraktikum

```
public interface KebutuhanPraktikum {  
    void alokasiAsistenLab ();  
  
    void cekPeralatanPraktikum();  
}
```

Gambar 13. Dokumentasi pada interface KebutuhanPraktikum

Untuk memahami alur kerja dari kode ini, berikut dijelaskan pembahasan kode dimulai dari kode pada class Mahasiswa:

```
public class Mahasiswa {  
    private String nim;  
    private String nama;  
    private String tipeJalur;  
  
    public Mahasiswa (String nim, String nama, String tipeJalur){  
        this.nim = nim;  
        this.nama = nama;  
        this.tipeJalur = tipeJalur;  
    }  
  
    public String getNim () {  
        return nim;  
    }  
    public String getNama() {  
        return nama;  
    }  
    public String getTipeJalur() {  
        return tipeJalur;  
    }  
}
```

Pada kode bagian class ini, berguna untuk mendefinisikan atribut apa saja yang dibutuhkan oleh objek Mahasiswa.

```
public interface MataKuliah {  
    String getKode();  
    String getNama();  
    int getSks();  
}
```

```
        boolean validasiPrasyarat (String nim);  
    }
```

Interface ini berguna sebagai cetakan kosong dasar dari mata kuliah yang akan diimplementasikan ke setiap jenis mata kuliah. Method yang ada harus diterapkan ke setiap class yang mengimplementasikannya. Berikut salah satu contoh penerapan interface tersebut:

```
public class MataKuliahKKN implements MataKuliah{  
    private String kode;  
    private String nama;  
    private int sks;  
  
    public MataKuliahKKN(String kode, String nama, int sks){  
        this.kode = kode;  
        this.nama = nama;  
        this.sks = sks;  
    }  
  
    @Override  
    public String getKode(){  
        return this.kode;  
    }  
    public String getNama(){  
        return this.nama;  
    }  
    public int getSks(){  
        return this.sks;  
    }  
    public boolean validasiPrasyarat(String nim){  
        System.out.println("Memeriksa prasyarat teori untuk mata kuliah " +  
this.nama);  
        return true;  
    }  
}
```

Di dalam class MataKuliahKKN ini mengimplementasikan interface MataKuliah. Pengimplementasian ini bersifat wajib untuk dilakukan.

```
public interface KebutuhanPraktikum {  
    void alokasiAsistenLab ();  
  
    void cekPeralatanPraktikum();  
}
```

Ini merupakan interface lain yang berisi method untuk kebutuhan dalam kegiatan praktikum. Sehingga khusus mata kuliah jenis praktikum wajib

mengimplementasikan interface ini sebagai cetakan awal. Berikut contoh penerapan yang dilakukan di dalam class MataKuliahPraktikum:

```
public class MataKuliahPraktikum implements KebutuhanPraktikum,
MataKuliah{
    private String kode;
    private String nama;
    private int sks;

    public MataKuliahPraktikum(String kode, String nama, int sks){
        this.kode = kode;
        this.nama = nama;
        this.sks = sks;
    }

    @Override
    public String getKode(){
        return this.kode;
    }
    public String getNama(){
        return this.nama;
    }
    public int getSks(){
        return this.sks;
    }
    public boolean validasiPrasyarat(String nim){
        System.out.println("Memeriksa prasyarat teori untuk mata kuliah " +
this.nama);
        return true;
    }

    public void alokasiAsistenLab(){
        System.out.println("Mengalokasikan asisten lab untuk praktikum " +
this.nama);
    }
    public void cekPeralatanPraktikum(){
        System.out.println("Memeriksa alat untuk praktikum " + this.nama);
    }
}
```

Di dalam class MataKuliahPraktikum ini dilakukan pengimplementasian dari interface MataKuliah yang merupakan cetakan dasar dari keseluruhan mata kuliah, dan interface KebutuhanPraktikum yang berisi method yang harus diimplementasikan di class.

Penggunaan interface ini dilakukan untuk memenuhi solid principle yang memisahkan setiap interface. Sehingga setiap class hanya mengimplementasikan method yang berhubungan langsung dengan tujuan dari dibentuknya class tersebut.

```
public interface PenghitunganUKT {  
    public double hitungUKT();  
}
```

Ini merupakan sebuah interface penghitungan dari ukt yang berguna sebagai cetakan dasar dan harus diinisialisasikan ke class yang mengimplementasikan interface ini. Penggunaan interface ini bertujuan agar setiap class yang memiliki aturan tersendiri dalam penghitungan ukt dapat menerapkan aturan tersebut di dalam class saja. Sehingga kode yang dibuat lebih mudah untuk dipahami.

Berikut adalah salah satu contoh pengimplementasian interface PenghitunganUKT ke sebuah class:

```
public class UKTInternasional implements PenghitunganUKT {  
    private double tarifInternasional;  
  
    public UKTInternasional(double tarifInternasional){  
        this.tarifInternasional = tarifInternasional;  
    }  
  
    @Override  
    public double hitungUKT () {  
        System.out.println("Bayar ukt untuk kelas internasional");  
        return this.tarifInternasional;  
    }  
}
```

Class ini berguna untuk menghitung nominal dari UKT yang mesti dibayarkan mahasiswa dengan tipe Internasional. Penghitungan nominal UKT ini akan berbeda dengan tipe jalur yang lainnya, berikut adalah contohnya:

```
public class UKTKaryawan implements PenghitunganUKT {  
    private double tarifJurusan;  
    private double biayaKelasMalam;  
  
    public UKTKaryawan(double tarifJurusan, double biayaKelasMalam){  
        this.tarifJurusan = tarifJurusan;  
    }  
  
    @Override  
    public double hitungUKT () {  
        System.out.println("Bayar ukt untuk kelas karyawan");  
        return this.tarifJurusan + this.biayaKelasMalam;  
    }  
}
```

Di dalam class UKTKaryawan ini dihitung dengan adanya penambahan biaya mengikuti kelas malam. Hal ini tidak sama dengan class dengan tipe internasional yang tidak memiliki tambahan biaya apapun.

Setelah itu, masuk ke class KRS:

```
import java.util.*;
```

```
public class KRS {  
    private List<MataKuliah> daftarMataKuliah;  
  
    public KRS(){  
        this.daftarMataKuliah = new ArrayList<>();  
    }  
  
    public void tambahMataKuliah(MataKuliah matkul){  
        this.daftarMataKuliah.add(matkul);  
    }  
  
    public List<MataKuliah> getDaftarMataKuliah(){  
        return this.daftarMataKuliah;  
    }  
}
```

Pada class ini berisi collection List dengan tipe MataKuliah yang berguna untuk menambahkan mata kuliah yang diikuti oleh seorang mahasiswa.

Berikut penjelasan alur kode dari class main (App):

```
Mahasiswa mhs1 = new Mahasiswa("105224028", "Neila Asynur",  
"REGULER");
```

Bagian ini merupakan penginisialisasian objek bertipe Mahasiswa dengan tipe jalur sebagai reguler.

```
KRS krsNeila = new KRS();  
krsNeila.tambahMataKuliah(new MataKuliahTeori("CS201", "Struktur Data",  
3));
```

```
System.out.println(" >> Memproses KRS untuk: " + mhs1.getNama());
```

Blok ini berguna untuk membuat objek krs dari mahasiswa yang bertipe KRS. Setelah objek krs mahasiswa dibuat, lakukan penambahan mata kuliah pada mahasiswa tersebut. Setelah menambahkan mata kuliah yang diambil oleh mahasiswa, berikan message yang akan ditampilkan ke layar pengguna sebagai pemberitahuan.

```
for (MataKuliah matkul : krsNeila.getDaftarMataKuliah()) {  
    matkul.validasiPrasyarat(mhs1.getNim());  
  
    if (matkul instanceof KebutuhanPraktikum) {  
        KebutuhanPraktikum prak = (KebutuhanPraktikum) matkul;  
        prak.cekPeralatanPraktikum();  
        prak.alokasiAsistenLab();  
    }  
}
```

}

Setelah itu, dilakukan pengecekan berlapis terhadap mata kuliah yang diambil oleh mahasiswa. Lakukan validasi prasayat terhadap mata kuliah yang diambil oleh mahasiswa. Kemudian melakukan pengecekan terhadap mata kuliah tersebut, jika mata kuliah merupakan bagian yang mengimplementasikan KebutuhanPraktikum maka harus dilakukan pengecekan alat praktikum dan pengalokasian asisten lab pada mata kuliah tersebut.

```
PenghitunganUKT uktNeila = new UKTReguler(8000000.0);
double totalUKTNeila = uktNeila.hitungUKT();
System.out.println("Total Biaya yang harus dibayar " + mhs1.getNama() +
" Rp" + totalUKTNeila);
System.out.println("-----
\n");
```

Ini merupakan blok yang berguna untuk menghitung nominal dari ukt yang harus dibayarkan oleh mahasiswa.

Dua objek mahasiswa lain merupakan contoh lain dalam pengimplementasian jenis nominal UKT yang harus dibayarkan oleh mahasiswa.

V. Kesimpulan

Kesimpulan dari praktikum di modul 11 ini adalah desain arsitektur menggunakan SOLID Principle dapat menyelesaikan masalah pembusukan kose pada sistem SIAKAD yang lama. Melalui prinsip Single Responsibility, kelas raksasa yang kaku telah dipecah menjadi komponen-komponen mandiri seperti Mahasiswa dan KRS yang hanya memiliki satu tanggung jawab. penerapan OCP terbukti sukses dalam menghilangkan pengkodisian if-else, sehingga penghitungan biaya ukt dapat dilakukan secara dinamis melalui paramater. Selain itu, OCP juga mempermudah dalam melakukan integrasi kurikulum baru lewat penambahan class tanpa memberikan resiko terhadap class lama. Interface KebutuhanPraktikum berhasil melindungi kelas non-praktikum dari error paksaan.

Sehingga, secara keseluruhan rekonstruksi ini menghasilkan arsitektur sistem akademis yang jauh lebih bersih, serta adaptif terhadap aturan bisnis di kampus ataupun pengembangan di masa depan.

VI. Daftar Pustaka

Tim Asisten Laboratorium Komputer. (2026). Modul Praktikum Pemrograman Berorientasi Objek - Modul 12: SOLID Principle. Program Studi Ilmu Komputer, Universitas Pertamina